

Unsupervised Learning: Clustering and Anomaly Detection

A worked summary with illustrations

Gentian Zavalani

17 September 2026

Notes written while working through the Machine Learning Specialization
(DeepLearning.AI, Coursera), Course 3: unsupervised learning

Abstract

These notes cover the two unsupervised algorithms: K -means clustering (Part I) and density-estimation anomaly detection (Part II). Every figure and every number in the tables was produced by running the algorithms on the datasets described, not drawn by hand. Each part ends with material the standard presentation tends to skip. For clustering, that means why the objective can never choose K for you and why one run of the algorithm is not enough; for anomaly detection, what the independence assumption costs, what feature shape costs, and where the arithmetic breaks. In practice these dominate the outcome.

1 What makes it unsupervised

Supervised learning is given pairs $(x^{(i)}, y^{(i)})$: inputs with the correct answer attached, from which it learns the map $x \mapsto y$. Unsupervised learning is given the inputs $x^{(1)}, \dots, x^{(m)}$ alone and must find structure in them. Two tasks:

- **Clustering**: partition the data into K groups of similar examples. Algorithm: K -means.
- **Anomaly detection**: learn what *normal* looks like, then flag anything that deviates from it.

One asymmetry worth noticing immediately. Anomaly detection *trains* on unlabelled data, but it *tunes* its one free parameter on a small labelled set. So it is unsupervised in the part that needs lots of data and supervised in the part that needs almost none. That is exactly the right split when positive examples are rare and expensive, and it is the reason the method exists at all.

Part I Clustering with K -means

find the groups

2 The algorithm

K -means partitions the data into K groups by alternating two steps until the assignments stop changing (Figure 1).

$$\text{assign: } c^{(i)} := \arg \min_{j \in \{1, \dots, K\}} \|x^{(i)} - \mu_j\|^2, \quad i = 1, \dots, m, \quad (1)$$

$$\text{move: } \mu_k := \frac{1}{|C_k|} \sum_{i \in C_k} x^{(i)}, \quad k = 1, \dots, K, \quad (2)$$

where $C_k = \{i : c^{(i)} = k\}$ is the set of examples currently assigned to centroid k , and $|C_k|$ is how many there are. The μ_k are the *centroids*; K is chosen in advance.

Clustering is used for market segmentation, grouping related news articles, organising DNA microarray data, grouping astronomical observations, and, as in Section 6, reducing the number of distinct colours in an image.

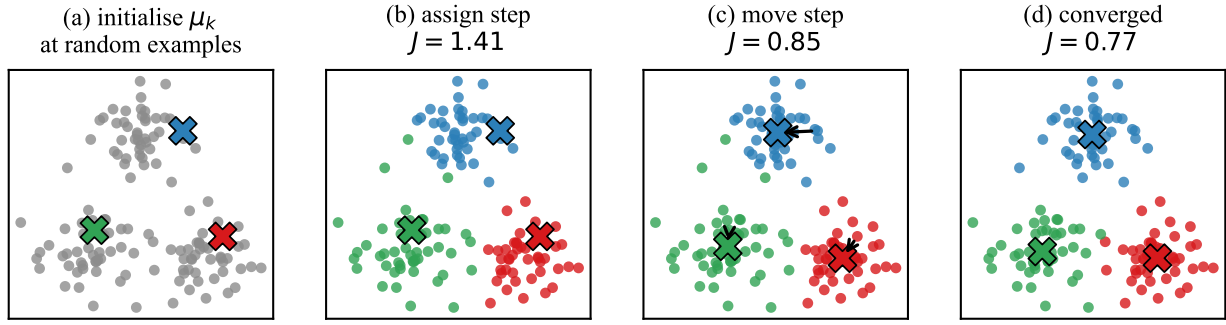


Figure 1: K -means on 135 points in three Gaussian blobs, $K = 3$. Crosses are centroids. Panel (b) recolours each point by (1); panel (c) slides each centroid to the mean of its own colour by (2), with arrows showing the displacement. Both steps decrease the distortion J , which is why the iteration converges. Note that panel (a) initialises the centroids at randomly chosen data points, not at random locations in space.

3 The objective, and why it converges

Both steps minimise the same quantity, the *distortion*

$$J(c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K) = \frac{1}{m} \sum_{i=1}^m \|x^{(i)} - \mu_{c^{(i)}}\|^2, \quad (3)$$

the mean squared distance from each example to the centroid it is assigned to. The two steps are exactly the two block minimisations of (3):

- the **assign** step minimises J over the $c^{(i)}$ with the μ_k held fixed. For each example independently the best choice is the nearest centroid, which is (1);
- the **move** step minimises J over the μ_k with the $c^{(i)}$ held fixed. The minimiser of a sum of squared distances to a set of points is their mean, which is (2).

So J is non-increasing at every half-step. It is bounded below by zero, and there are only finitely many possible assignments, so the algorithm terminates in a finite number of iterations. This is coordinate descent, not gradient descent, and it has no learning rate to tune.

The single most useful debugging check

J must **never increase**, not even slightly, at any half-step. If your implementation shows it rising, you have a bug, almost always a swapped axis in the distance computation (`axis=1` where you meant `axis=0`) or centroids updated from the previous iteration's assignments. Print J after every step while developing.

4 Local optima, and why you run it many times

Termination is guaranteed; *optimality is not*. K -means converges to a local minimum of (3), and which one depends entirely on the initialisation. Figure 2 shows the standard failure: one true cluster gets split between two centroids while two other clusters are merged under a third. The result is stable, in the sense that no single reassignment improves it, and it is more than four times worse than the global optimum.

The fix is to run the whole algorithm many times and keep the best result:

1. For $t = 1, \dots, T$ (with T typically 50 to 1000): initialise $\mu_1, \dots, \mu_K$ at K randomly chosen distinct training examples, run K -means to convergence, and record the final J .
2. Keep the clustering with the lowest J .

Initialising at actual data points matters: it guarantees every centroid starts somewhere the data actually are, so no cluster is empty at the first assign step. If a cluster does become empty later, which makes (2) a division by zero, either drop that centroid and continue with $K - 1$, or reinitialise it at a random example.

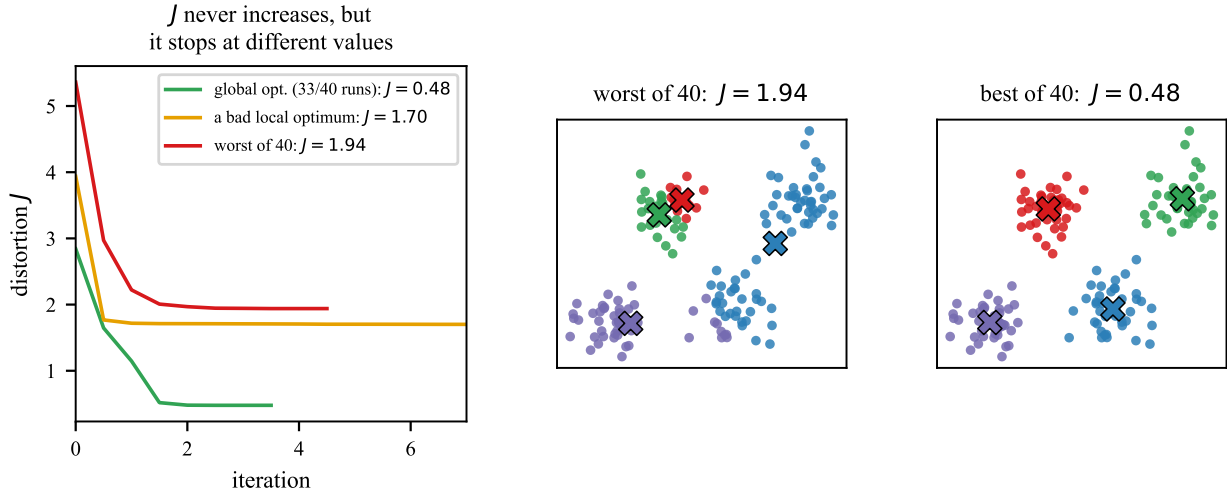


Figure 2: Forty random initialisations on four well-separated clusters. Left: J against iteration for three of them; every curve is monotone, but they level off at different values. Middle and right: the worst and best outcomes. Thirty-three of the forty runs reached $J = 0.48$; the remaining seven got stuck between 1.70 and 1.94. Taking the best of many restarts is not a refinement, it is a requirement.

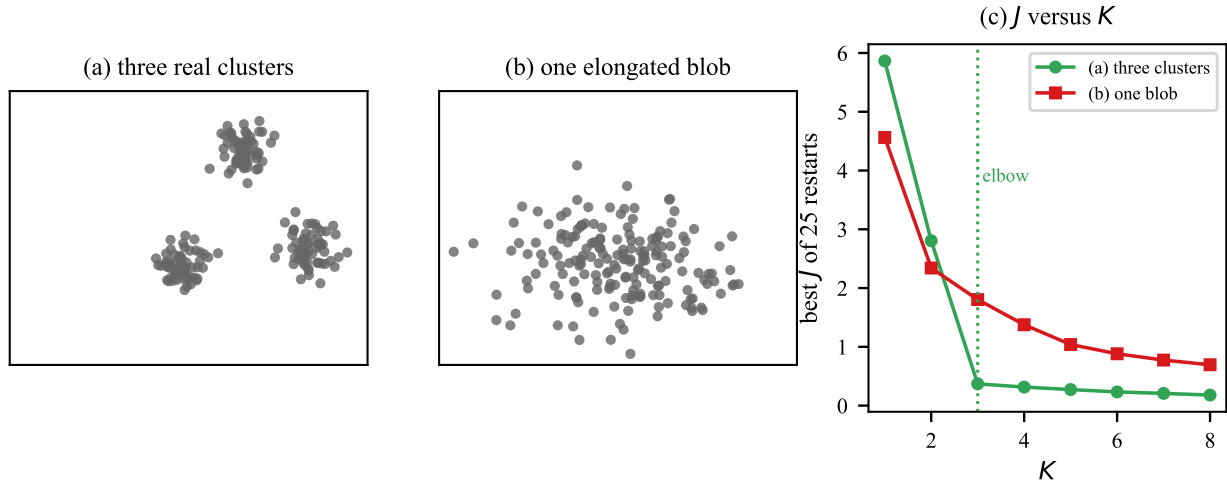


Figure 3: The elbow method, and its failure mode. (c) plots the best J over 25 restarts against K for both datasets. The three-cluster data give a sharp elbow at $K = 3$; the single blob gives a smooth curve with no distinguished point anywhere.

5 Choosing K

There is no value of K that (3) will pick out for you, and this is not a gap in the method but a property of the objective: J decreases monotonically as K grows, reaching zero when $K = m$ and every example is its own cluster. So J alone can only ever recommend $K = m$.

The *elbow method* plots the best J against K and looks for the point where the curve stops dropping sharply. Figure 3 shows both why this is appealing and why it is unreliable. On data with genuine well-separated clusters there is a sharp elbow at the right K . On data that is simply one elongated blob, which is extremely common, the curve decreases smoothly and no elbow exists to find. Reporting an elbow from such a curve is reading a structure into the data that is not there.

The more honest approach is to choose K by what the clusters are *for*. If you are sizing T-shirts, the question is not which K minimises a sum of squares but whether the extra manufacturing and inventory cost of five sizes is repaid by the better fit relative to three. If you are segmenting a market, K is bounded by how many distinct campaigns you can actually run. Evaluate K on the downstream objective, not on J .

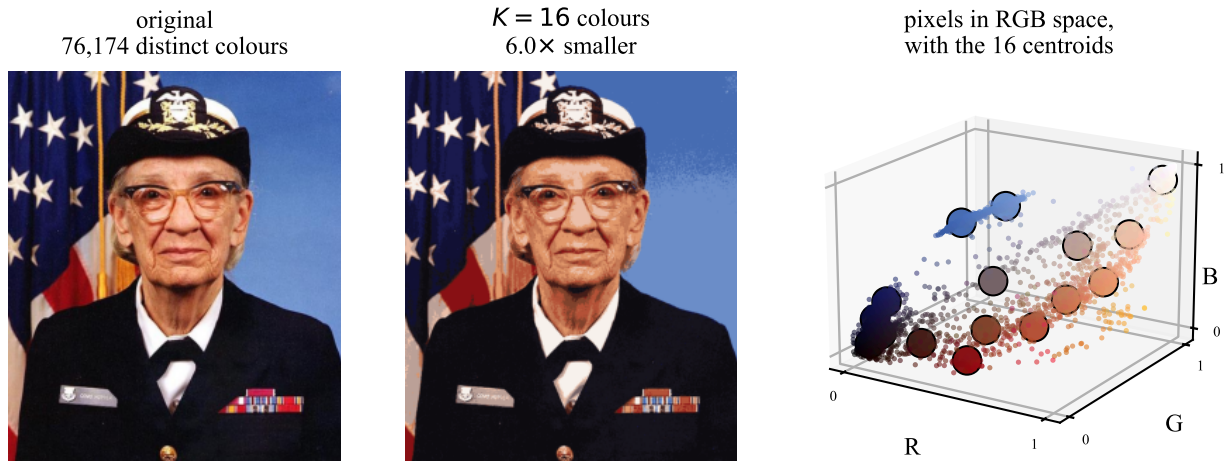


Figure 4: Colour quantisation by K -means, $K = 16$. The photograph contains 76,174 distinct colours; the reconstruction uses sixteen and remains clearly legible, with visible banding in the smooth background where the palette is too coarse. Right: a subsample of the pixels plotted in RGB space, each drawn in its own colour, with the sixteen centroids. The clusters are the natural colour families of the image: skin tones, navy uniform, red and white flag.

6 Application: image compression

A photograph stores 24 bits per pixel: one byte each for red, green and blue, so $2^{24} \approx 16.8$ million representable colours. Treat each pixel as a point $x^{(i)} \in [0, 1]^3$ and run K -means with $K = 16$ on those points. The 16 centroids are the 16 colours that best represent the image in the least-squares sense of (3), and each pixel is then stored as the 4-bit index of its nearest centroid (Figure 4).

The arithmetic, for the 512×600 image shown:

$$\begin{array}{ll}
 \text{original} & 307,200 \text{ pixels} \times 24 \text{ bits} = 7,372,800 \text{ bits,} \\
 \text{compressed} & \underbrace{307,200 \times 4 \text{ bits}}_{\text{one index per pixel}} + \underbrace{16 \times 24 \text{ bits}}_{\text{the palette}} = 1,229,184 \text{ bits,}
 \end{array}$$

a factor of 6.0. This is *lossy* compression: the original colours cannot be recovered, and (3) is precisely the mean squared reconstruction error you are choosing to accept. Note also the practical shortcut used here. The centroids were fitted on a random subsample of 8,000 pixels rather than all 307,200, because the centroid of a colour family is estimated perfectly well from a sample, and only the final assignment needs to touch every pixel.

Part II Anomaly detection by density estimation

find the things that do not belong

7 Anomaly detection: the algorithm

Fit a density $p(x)$ to data that is unlabelled and almost entirely normal. For a new example x_{test} , compute $p(x_{\text{test}})$ and

$$\text{flag } x_{\text{test}} \text{ as anomalous} \iff p(x_{\text{test}}) < \varepsilon. \quad (4)$$

That is the whole algorithm (Figure 5). The motivating case is manufacturing: after an aircraft engine comes off the line you measure features such as heat generated (x_1) and vibration intensity (x_2). A manufacturer produces very few bad engines, so collecting normal examples is easy and collecting defective ones is not. The technique is called *density estimation*, and the same pattern appears in fraud detection (login frequency, pages visited, transactions, typing speed), data-centre monitoring (memory, disk accesses per second, CPU load, and ratios such as CPU load over network traffic), and telecom cell-tower monitoring.

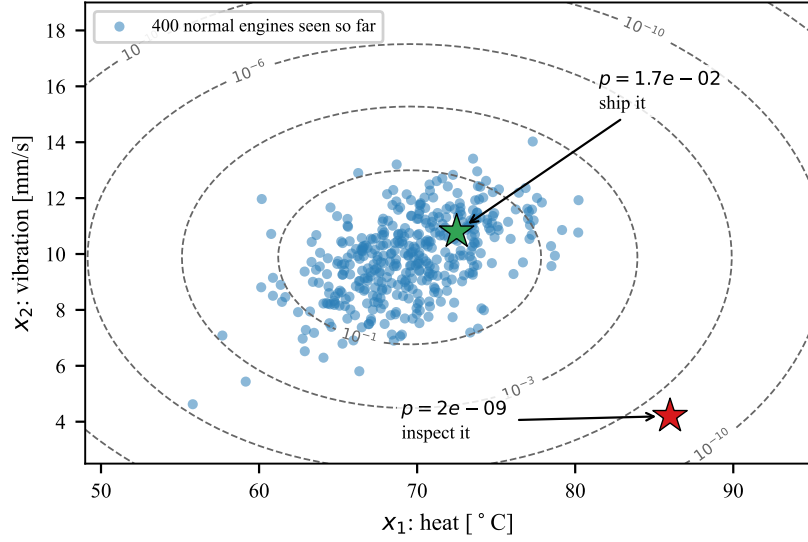


Figure 5: The decision rule (4) on 400 normal engines. Contours are labelled by how far p has fallen below its peak. The green engine sits where the training data are dense and is passed; the red one sits where p is seven orders of magnitude smaller and is sent for inspection. Note that in fraud detection the response to a flag is *not* to close the account. It is to escalate: ask the security team to look, request identity verification, or serve a CAPTCHA. The detector buys attention, not a verdict.

The three steps

1. Choose features x_j that you think might take unusually large or small values for an anomalous example.
2. Fit the parameters. For each feature $j = 1, \dots, n$,

$$\mu_j = \frac{1}{m} \sum_{i=1}^m x_j^{(i)}, \quad \sigma_j^2 = \frac{1}{m} \sum_{i=1}^m (x_j^{(i)} - \mu_j)^2. \quad (5)$$

3. For a new x , compute

$$p(x) = \prod_{j=1}^n p(x_j; \mu_j, \sigma_j^2) = \prod_{j=1}^n \frac{1}{\sqrt{2\pi} \sigma_j} \exp\left(-\frac{(x_j - \mu_j)^2}{2\sigma_j^2}\right) \quad (6)$$

and apply (4).

Two details. The variance in (5) is divided by m , not $m - 1$: this is the maximum-likelihood estimate, and it is what the graded exercises check. The difference is a factor $m/(m - 1)$, about 0.3% at $m = 307$, and gets absorbed entirely into the choice of ε .

Second, the product in (6) assumes the features are statistically independent. This is essentially always false, and the method works anyway, but the assumption has a sharp geometric consequence, which is the subject of Section 11.

8 Evaluating it, and choosing ε

You cannot choose ε from the training data, because it contains no anomalies: nothing in it indicates where “too improbable” begins. So keep three splits:

split	contents	purpose
train	normal only, unlabelled	fit μ_j, σ_j^2
validation	mostly normal + a few labelled anomalies	choose ε
test	same mixture	report the score, <i>once</i>

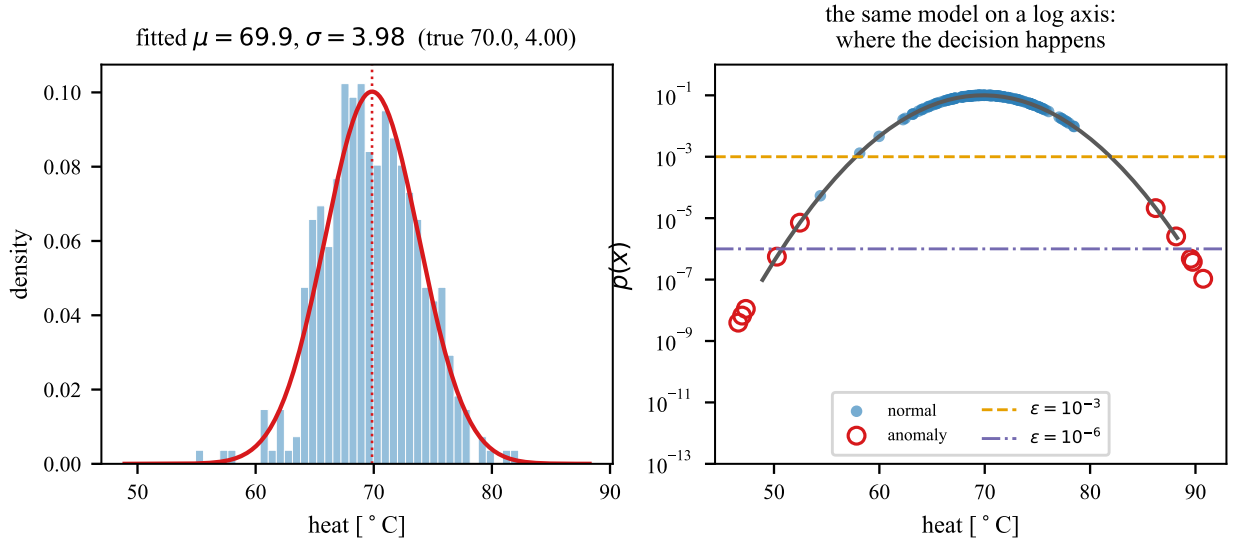


Figure 6: A single feature, so the whole model is visible. Left: 400 normal engines and the density fitted by (5). Right: the same density on a logarithmic axis, with 160 validation examples scored by it. The left panel is what the model believes; the right panel is what you make decisions with. On a linear density axis the entire anomaly region is pinned indistinguishably against zero, while on a log axis it spreads over ten orders of magnitude, and every thresholding decision lives in exactly that range.

Never use accuracy

On a validation set that is 7% anomalous, a detector that flags *nothing* scores 93% accuracy. In real fraud or manufacturing settings anomalies are 0.1% or less, and the same do-nothing detector scores 99.9%. Accuracy cannot distinguish a working detector from a switched-off one, so it gives the search for ε nothing to climb.

Use the confusion matrix instead (Figure 7), and see Figure 8 for what the sweep over ε looks like:

$$\text{prec} = \frac{tp}{tp + fp}, \quad \text{rec} = \frac{tp}{tp + fn}, \quad F_1 = \frac{2 \text{prec} \cdot \text{rec}}{\text{prec} + \text{rec}}. \quad (7)$$

Note that tn appears nowhere in (7). That is precisely why these quantities survive severe class imbalance: they refuse to take credit for the enormous pile of normal examples you correctly ignored.

F_1 is the *harmonic* mean, which matters. Precision 1.00 paired with recall 0.02 has arithmetic mean 0.51, which looks respectable, and $F_1 = 0.039$, which is correctly damning. Any usable metric here must punish that combination, because “flag the single most improbable example and nothing else” achieves perfect precision while being useless.

F_1 is a choice, not a law

You are picking a point on a trade-off curve, not finding a correct answer. F_1 weights precision and recall equally, which is a specific and often wrong opinion. For aircraft engines a missed defect costs vastly more than a needless inspection, so the better objective is to maximise recall subject to a false-alarm budget you can actually staff, say twenty inspections a day. That formulation also needs no labels, which is often decisive.

9 Anomaly detection versus supervised learning

Both can be pointed at the same labelled data, so the choice needs a criterion. It is not simply how many positives you have; it is what *kind* of positives to expect next.

	truly anomalous	truly normal
flagged	<i>tp</i> caught it	<i>fp</i> false alarm
not flagged	<i>fn</i> missed it	<i>tn</i> never used

$\text{precision} = \frac{tp}{tp + fp}$ (columns of the top row)
 $\text{recall} = \frac{tp}{tp + fn}$ (rows of the left column)

Figure 7: The four outcomes of a flagging decision. Only three of the cells appear in (7): the count of correctly ignored normal examples, *tn*, is deliberately discarded.

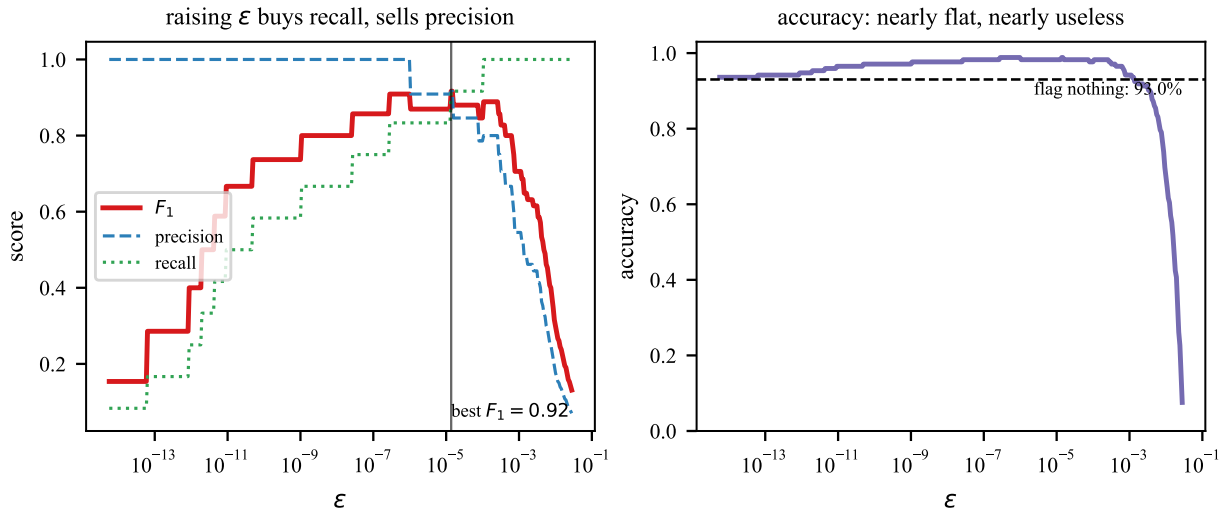


Figure 8: Sweeping ϵ over the full twelve decades spanned by p on the validation set. As ϵ grows the flagged set only ever grows, so recall rises monotonically and precision falls monotonically; F_1 rises, peaks, and falls, and the search finds that peak. Accuracy, meanwhile, is nearly flat at the base rate across the entire sweep.

Use anomaly detection	Use supervised learning
Very few positive examples (0–20 is typical); many negatives.	Plenty of both positive and negative examples.
Many <i>different</i> kinds of anomaly, and the next one may look nothing like anything you have seen.	Future positives are likely to resemble the positives in your training set.
<i>Examples:</i> fraud, manufacturing defects, machine and infrastructure failures.	<i>Examples:</i> spam filtering, weather prediction, disease classification.

The logic: if the algorithm cannot learn what a positive looks like from examples, because the next defect will be a genuinely new kind of defect, then do not try. Model normality instead, and flag departure from it. A supervised classifier trained on the defects you happened to see will confidently pass a defect of a kind it has never met; a density model will not.

10 Feature engineering beats model choice

Make each feature roughly Gaussian

Histogram every feature before modelling it. If one is skewed, transform it: $\log x$, $\log(x + c)$, $\sqrt{x}$, $x^{0.3}$ are the usual candidates. Apply the *same* transform to the training, validation and test sets.

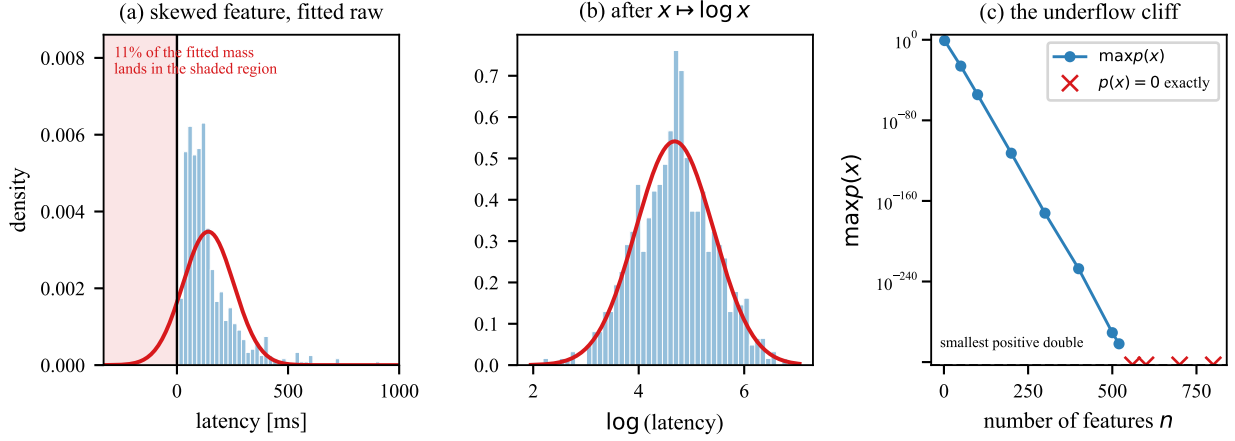


Figure 9: (a) A Gaussian fitted to a log-normal feature: the mean is dragged above the mode, the variance inflates to cover the tail, and 11% of the fitted mass falls on physically impossible negative values. (b) The same feature after $x \mapsto \log x$; the skewness drops from 2.24 to -0.02 . (c) The product (6) decays like $e^{-1.42n}$ for standard normal data and reaches the smallest positive double near $n = 520$. Beyond that, $p(x)$ is exactly 0.0 for every example.

Figure 9(a) shows why this is not cosmetic. The feature is a log-normal latency, and it carries **no anomaly signal whatsoever**. Anomalous servers in this dataset were drawn from exactly the same latency distribution as healthy ones; the signal lives entirely in a second, well-behaved feature. Fitting a Gaussian to it directly inflates the variance to cover the legitimate right tail, which places 11% of the model’s probability mass on *negative* latency. Honest slow-but-fine servers then land four standard deviations out and outrank the genuinely broken ones.

features	validation F_1	precision	recall	test F_1
latency as given	0.750	0.609	0.933	0.737
log(latency)	1.000	1.000	0.933	0.966

Recall is untouched; precision moves from 0.609 to 1.000. **Bad feature shape costs you precision, not recall**, and a detector that cries wolf gets switched off, which is operationally identical to not having one.

Error analysis, and combination features

When an anomaly slips through with a large $p(x)$, look at that example and ask what new feature would have distinguished it. Often the answer is a *ratio* or product of existing features: neither CPU load nor network traffic is individually unusual, but their ratio is. Adding CPU/network as its own column lets the product model (6) express a relationship it otherwise structurally cannot.

Work in $\log p$, not p

Figure 9(c) is a failure with no statistical content at all. Once n exceeds roughly 520, every $p(x)$ underflows to exactly 0.0, so $p(x) < \varepsilon$ is true for *everything* and the detector flags 100% of your data, while looking like a badly tuned threshold rather than an arithmetic problem. Evaluate

$$\log p(x) = -\frac{1}{2} \sum_{j=1}^n \left[\frac{(x_j - \mu_j)^2}{\sigma_j^2} + \log \sigma_j^2 + \log 2\pi \right]$$

instead, and the failure mode does not exist: a sum of logs has no cliff. For the same reason, space the search grid for ε uniformly in $\log p$ rather than in p . A grid uniform in p places nearly all of its nodes in the top decade, where nothing is being decided, and resolves the tail with one or two points.

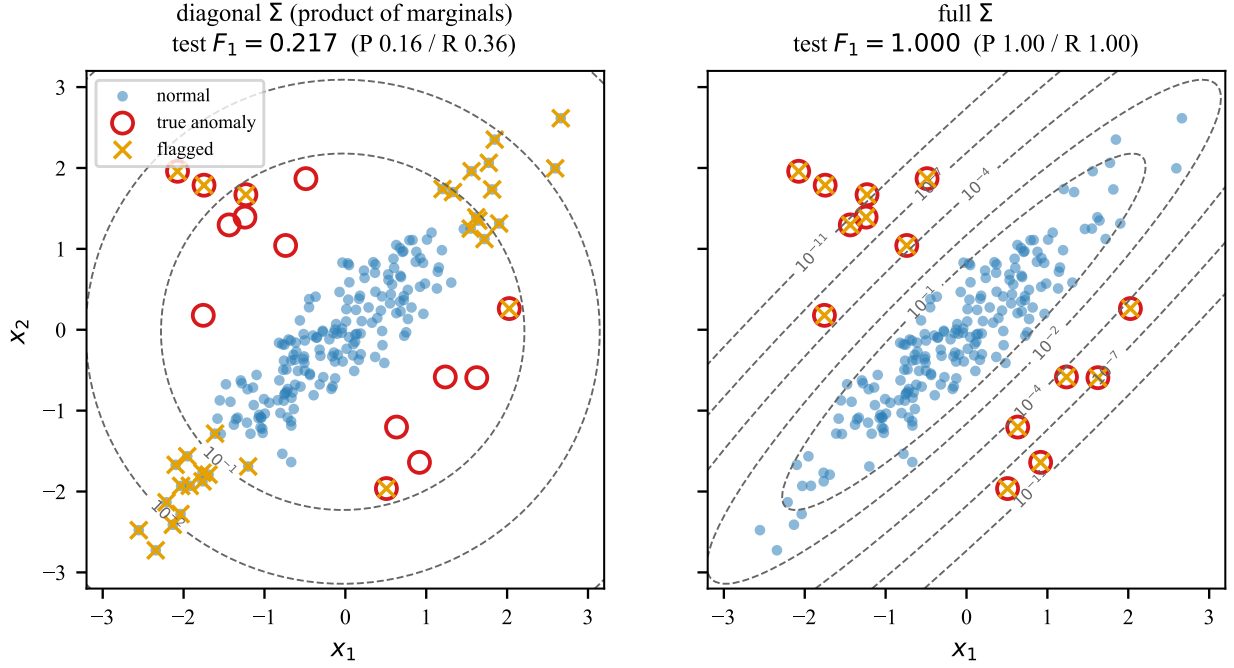


Figure 10: Identical data, identical machinery; the only difference is whether Σ may have off-diagonal entries. Left: the diagonal model’s contours are nearly circular, they swallow the anomalies whole, and the threshold that best fits the validation set flags normal points at the corners instead. Right: the full-covariance contours follow the ridge and every anomaly falls outside them. Test F_1 goes from 0.217 to 1.000.

11 What the independence assumption costs

Because (6) is a product of per-feature densities, its level sets are **axis-aligned ellipses**. The model can therefore only ever notice “this one coordinate is extreme”. An example that is perfectly ordinary in *every individual feature* while being impossible as a *combination* is invisible to it.

Figure 10 is built to make this quantitative. The data have correlation 0.93, and the anomalies were chosen so that every one of them lies within 2.1 standard deviations of the mean in *every* coordinate, while the most extreme *normal* point reaches 2.63. Marginally, then, the anomalies are *less* extreme than ordinary data, and no per-feature rule, whether a *z*-score cut, a bounding box or a product of marginals, can separate them at any threshold.

The fix is the **multivariate Gaussian**, which replaces the n separate variances by a full covariance matrix:

$$p(x; \mu, \Sigma) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu)^\top \Sigma^{-1}(x - \mu)\right), \quad \Sigma = \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \mu)(x^{(i)} - \mu)^\top. \quad (8)$$

Its contours can tilt, so correlations are captured automatically. The original model (6) is exactly (8) with Σ constrained to be diagonal.

	independent (product)	multivariate Gaussian
parameters	$2n$	$n + n(n + 1)/2$
correlations	only via hand-built features	automatic
data needed	works at $m \approx n$	needs $m > 10n$; Σ singular if $m < n$
cost	$O(n)$ per example	$O(n^3)$ once, $O(n^2)$ per example

Computing the multivariate density properly

Do not form Σ^{-1} or $|\Sigma|$. Take the Cholesky factor $\Sigma = LL^T$ once; then solving $Lw = x - \mu$ gives $w^T w = (x - \mu)^T \Sigma^{-1} (x - \mu)$, and $\log |\Sigma| = 2 \sum_i \log L_{ii}$ comes free. One factorisation, no explicit inverse, no overflow in the determinant, and better conditioning.

12 A worked case: the two fixes are not independent

To close, a six-feature problem where both of the preceding sections are needed at once. The data are telemetry from an iterative linear solver: estimated condition number, iteration count, final residual, residual decay rate, preconditioner setup time, peak memory. Problem size and conditioning are *latent* rather than columns, so they induce genuine correlation between the columns you do observe. Three failure modes are planted: stagnation at the iteration cap; preconditioner degradation, where a run needs about twice the iterations its conditioning predicts; and a silently easier problem, where the operator came out far better conditioned than the mesh warrants. Recall is reported per mode, because the aggregate hides everything: each mode is only 8 of 424 examples.

model	test F_1	recall by failure mode		
		stagnation	precond. degrad.	easier problem
raw + diagonal Σ	0.345	0.62	0.00	0.00
raw + full Σ	0.345	0.62	0.00	0.00
log + diagonal Σ	0.744	1.00	0.00	1.00
log + full Σ	0.958	1.00	1.00	0.88
log + residual feature + diagonal Σ	0.941	1.00	1.00	1.00

The first two rows are the point. Giving the model a full covariance matrix on the raw features buys **exactly nothing**. The real dependence here is

$$\text{iters} \approx a + b \log_{10}(\text{cond}) + \text{noise},$$

which is not linear in the raw coordinates: the raw correlation between `cond` and `iters` is +0.430, and after the transform it is +0.876. Covariance can only capture *linear* structure, so the transform is a *precondition* for the covariance model, not an alternative to it. The two fixes are not independent improvements between which one chooses.

The last row is the other lesson. Regressing `iters` on $\log_{10}(\text{cond})$ on the training set and supplying the residual as a single extra feature hands the *cheap diagonal* model the same conditional information the full covariance had to discover, and reaches perfect recall on all three modes with $O(n)$ parameters instead of $O(n^2)$. It also produces something you can hand to whoever gets paged: *this run took 2.1 times the iterations its conditioning predicts*. Reach for a richer model when you cannot name the structure; when you can name it, build the feature.

Where the two parts meet

The single Gaussian of (6) assumes normality has *one* centre. If your normal data are genuinely multimodal (two populations of machine, or daytime and overnight traffic), the fitted Gaussian puts its peak in the empty space *between* the modes and assigns low density to both, so it flags healthy examples from each. The repair is Part I: cluster the training data first, then either fit one density per cluster, or use the distance to the nearest centroid in place of $p(x)$. Clustering finds where the data are; density estimation says how surprised to be at a new point. They compose.

If you remember seven things

1. (***K*-means**) Assign to the nearest centroid, move each centroid to the mean of its own points. Both steps minimise the same distortion J . It must therefore never increase, which is your best debugging check.
2. (***K*-means**) Convergence is to a *local* optimum. Restart from many random data points and keep the lowest J ; here seven runs in forty got stuck at four times the best value.
3. (***K*-means**) J falls monotonically with K , so it can never choose K for you. The elbow is often absent. Choose K by what the clusters are for.
4. Fit $p(x)$ on normal data, flag when $p(x) < \varepsilon$. That is the entire anomaly-detection algorithm.
5. Tune ε on a small labelled validation set using F_1 or a false-alarm budget, never accuracy.
6. Use anomaly detection when future anomalies will not resemble past ones; use supervised learning when they will.
7. Histogram every feature and transform the skewed ones; this buys more than any change of model. And remember the product form assumes independence, which forces axis-aligned contours. Fix that with engineered combination features or a full covariance matrix, transforming first, because covariance sees only linear structure.

Every figure and table in these notes was produced by running the algorithms on the stated datasets. The implementations are in `kmeans/` and `anomaly-detection/` in this repository; the figures are generated by `notes/make_figs.py`.